



Lab Title: Web Architecture Security and XSS Vulnerability Identification

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: Web Application Security Essential

Instructor Name: Ahmed Bukar

Date of Submission: 10/11/2025

Executive Summary

This lab focused on understanding how web applications function through a three-tier architecture and identifying common security weaknesses — specifically, Cross-Site Scripting (XSS) vulnerabilities. By building and testing both vulnerable and secure versions of login and search web pages, the lab demonstrated how improper handling of user input allows attackers to inject and execute malicious scripts. Students explored how browsers interpret HTML, JavaScript, and user data, gaining practical insight into how security flaws can compromise confidentiality, integrity, and user trust.

Through comparative testing, it was observed that unsafe coding practices like using `innerHTML` with unvalidated input expose applications to XSS attacks, while secure practices — such as output encoding, using `textContent`, and sanitizing input — prevent malicious script execution. The lab also reinforced the concept that client-side security alone is insufficient and that protection must exist at every layer of the web stack (client, server, and database).

Overall, this exercise provided a hands-on understanding of how vulnerabilities emerge, how they are exploited, and how secure coding principles can mitigate these threats. By mastering the data flow between client, server, and database, participants learned to design safer web applications and implement layered defenses against XSS and similar injection-based attacks.

Lab Objective

By completing this lab, you will be able to:

1. Understand Web Architecture

- Explain how the client, server, and database interact in a three-tier web application model.
- Describe the flow of web requests and responses using real-world examples (e.g., Facebook login).

2. Identify Security Vulnerabilities

- Recognize XSS vulnerabilities caused by improper input handling.
- Observe how untrusted input can execute arbitrary code when inserted into the DOM.

3. Implement Secure Coding Practices

- Use proper input sanitization and output encoding to prevent XSS.
- Replace unsafe methods (like `innerHTML`) with secure alternatives (`textContent`, `escapeHTML`).

4. Compare Vulnerable and Secure Implementations

- Build and test both insecure and secure versions of login and search pages.
- Document and analyze the behavioral differences between them.

5. Apply Defense-in-Depth Principles

- Understand the importance of validating data on both client and server sides.
- Explain how layered security (client, server, and database) helps prevent multi-step attacks such as session hijacking.

Lab Exercises

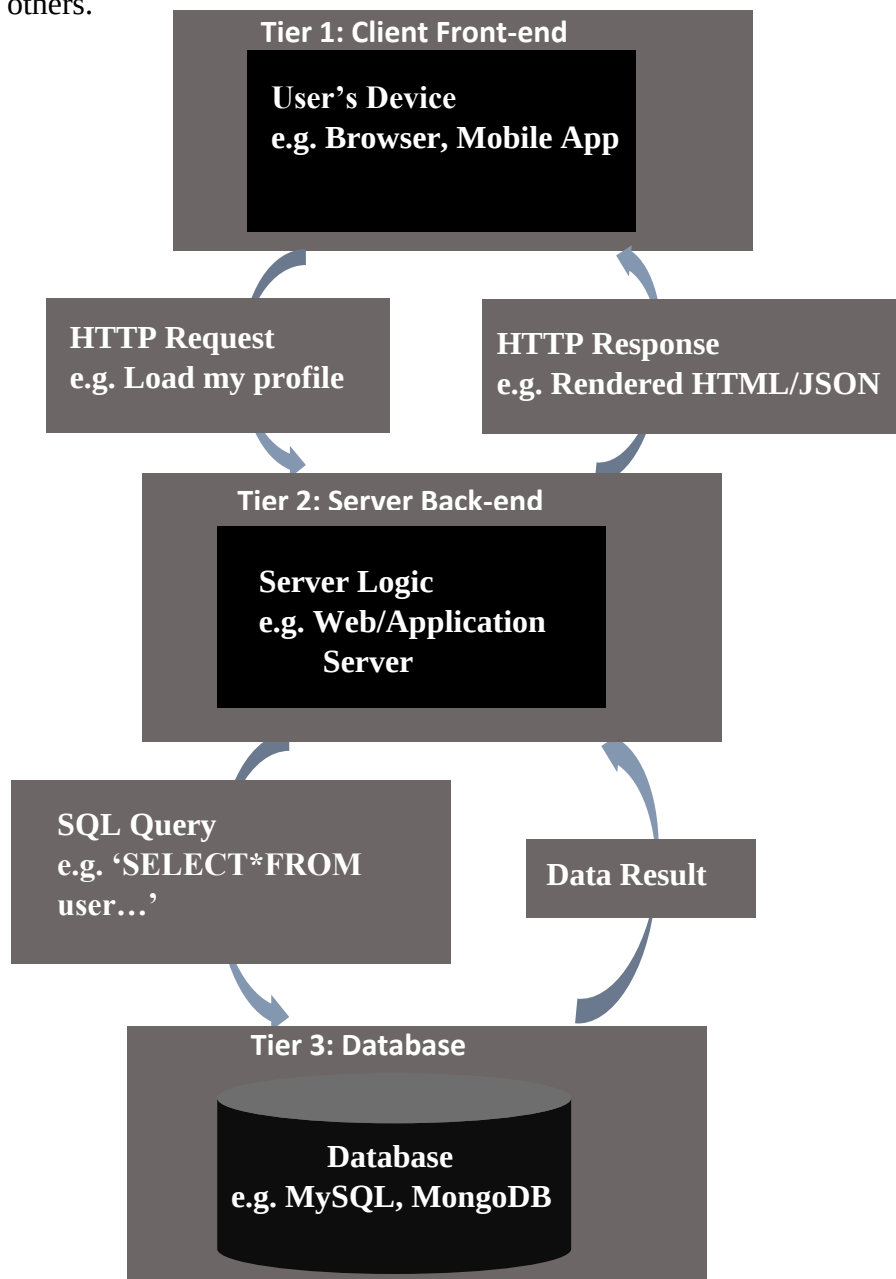
Exercise 1: Understanding Web Architecture

Step 1: Visualizing three-Tier of Web Architecture

The three-tier web architecture is a structured model that separates a web application into three main layers - the **presentation tier**, the **application (logic) tier**, and the **data tier**.

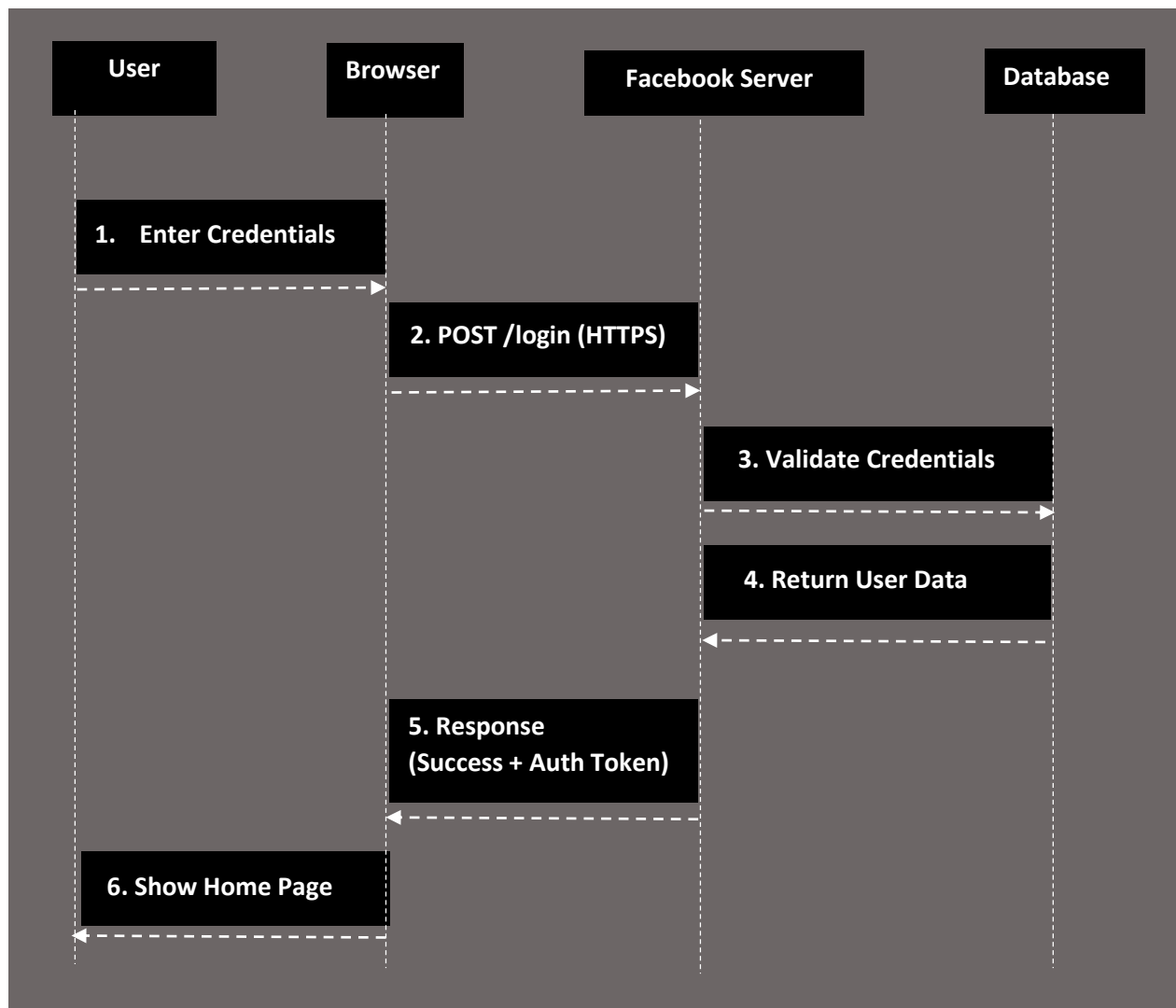
- The **Presentation Tier (Client/UI)** is the front end that users interact with through a web browser. It handles input and displays results.
- The **Application Tier (Server/Logic Layer)** processes user requests, applies business rules, and communicates between the client and the database.
- The **Data Tier (Database Layer)** securely stores and manages data, accessible only through the application tier.

Visualizing this architecture helps illustrate how each layer works independently yet communicates through defined interfaces. This layered design improves security, scalability, and maintainability, since each tier can be managed or updated separately without affecting the others.



VISUALIZING THE FLOW: YOU (Client) → INTERNET → FACEBOOK SERVER → FACEBOOK DATABASE

This sequence diagram starts when a user visits a login page. The user enters their username and password and hits the login button. The login page sends a message to the database with the user login data to be checked. Once the database has checked the username and password, the database returns the user's data (e.g. user ID, name, preferences) to the server. If successful, the server generates a session or authentication token (like a key) and sends it back to the browser, along with a redirect to the user's home page. If the credentials are invalid, the server sends an error response, the browser receives the response. On success, it stores the authentication token (often in a cookie) and loads the user's Facebook homepage. The user is now logged in.



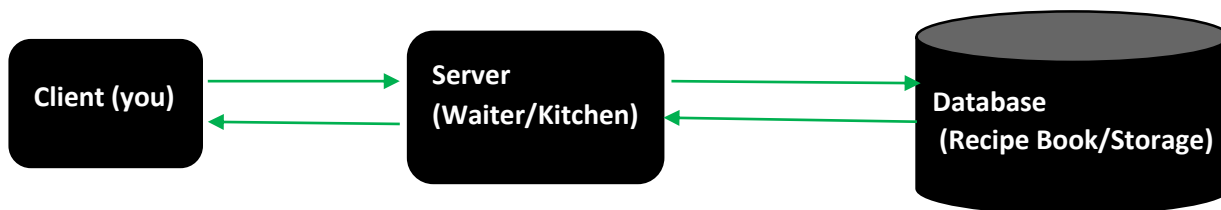
Step 2: Restaurant Analogy Mapping

This diagram shows the web architecture component equivalent to a restaurant

- Client (You)= The customer ordering food
- Server (Waiter/Kitchen) = Takes order and prepares food
- Database (Recipe Book/Storage) = Where all the ingredients and recipes are kept

Discussion:

It is dangerous if the "customer" can directly access the "recipe book" because, in web application terms, it means the client (user) can directly communicate with the database without going through the server. This would allow unauthorized users to view, modify, or delete sensitive data, leading to data breaches and system compromise. The server acts as a security barrier that validates requests, enforces permissions, and protects the database from malicious actions. Without this layer, attackers could perform SQL injections, steal confidential information, or corrupt critical data. Therefore, direct client access to the database removes essential security controls and exposes the entire system to serious risks.



Exercise 2: Creating Vulnerable Web Applications

Step 1: I reviewed and used the secure login form provided in the course materials. The code included structured HTML and CSS that demonstrated secure design practices for building login interfaces, such as styling and layout for clarity and usability.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vulnerable Login - ICDF Lab</title>
  <style>
    .vulnerable {
      background: #ffe6e6;
      padding: 20px;
    }
  </style>
</head>
<body>
  <div class="vulnerable">
    <h1>Vulnerable Login</h1>
  </div>
</body>
</html>
```

```
    max-width: 500px;
    margin: 50px auto;
    border: 3px solid red;
}

.warning-text {
    color: red;
    font-weight: bold;
}

input {
    box-sizing: border-box;
    width: 100%;
    padding: 10px;
    margin: 10px 0;
    border: 1px solid #ccc;
}

button {
    width: 100%;
    padding: 10px;
    background: red;
    color: white;
    border: none;
    cursor: pointer;
    font-size: 16px;
}

button:hover {
    background: darkred;
}

#message {
    margin-top: 20px;
    min-height: 50px;
    padding: 15px;
    background: white;
    border: 1px solid #ddd;
}

.message-content {
    background: lightyellow;
    padding: 15px;
}

</style>
```

```

</head>
<body>
  <div class="vulnerable">
    <h1>VULNERABLE LOGIN FORM</h1>
    <p class="warning-text">WARNING: This form has XSS vulnerabilities!</p>

    <form id="loginForm">
      <input type="text" id="username" placeholder="Username">
      <input type="password" id="password" placeholder="Password">
      <button type="button" id="loginBtn">
        Login (Insecure)
      </button>
    </form>

    <div id="message">
      Messages will appear here...
    </div>
  </div>

  <script>
    // Vulnerable function (intentionally insecure for testing)
    function vulnerableLogin() {
      const username = document.getElementById('username').value;
      const messageDiv = document.getElementById('message');

      // VULNERABLE: Direct user input placed into innerHTML (this is where
XSS happens)
      messageDiv.innerHTML = `
        <div class="message-content">
          <h4>Welcome back, ${username}!</h4>
          <p>Your login was successful.</p>
          <p>Last login: ${new Date().toLocaleString()}</p>
        </div>
      `;
    }

    // Attach event listeners and initialize after DOM is ready
    document.addEventListener('DOMContentLoaded', function () {
      // Attach click handler
      document.getElementById('loginBtn').addEventListener('click',
vulnerableLogin);

      // Prevent form submission if user presses Enter
      document.getElementById('loginForm').addEventListener('submit',
function (e) {

```

```
e.preventDefault();  
vulnerableLogin();  
});  
});  
</script>  
</body>  
</html>
```

vulnerable Login - ICDFA Lab



File C:/Users/1/OneDrive/Desktop/vulnerable-login.html

VULNERABLE LOGIN FORM

WARNING: This form has XSS vulnerabilities!

Login (Insecure)

Welcome back, !

Your login was successful.

Last login: 11/9/2025, 1:07:21 AM

This page says

XSS

OK

WARNING: This form has XSS vulnerabilities!

``

.....

Login (Insecure)

Welcome back, !

Your login was successful.

Last login: 11/9/2025, 4:01:23 AM

VULNERABLE LOGIN FORM

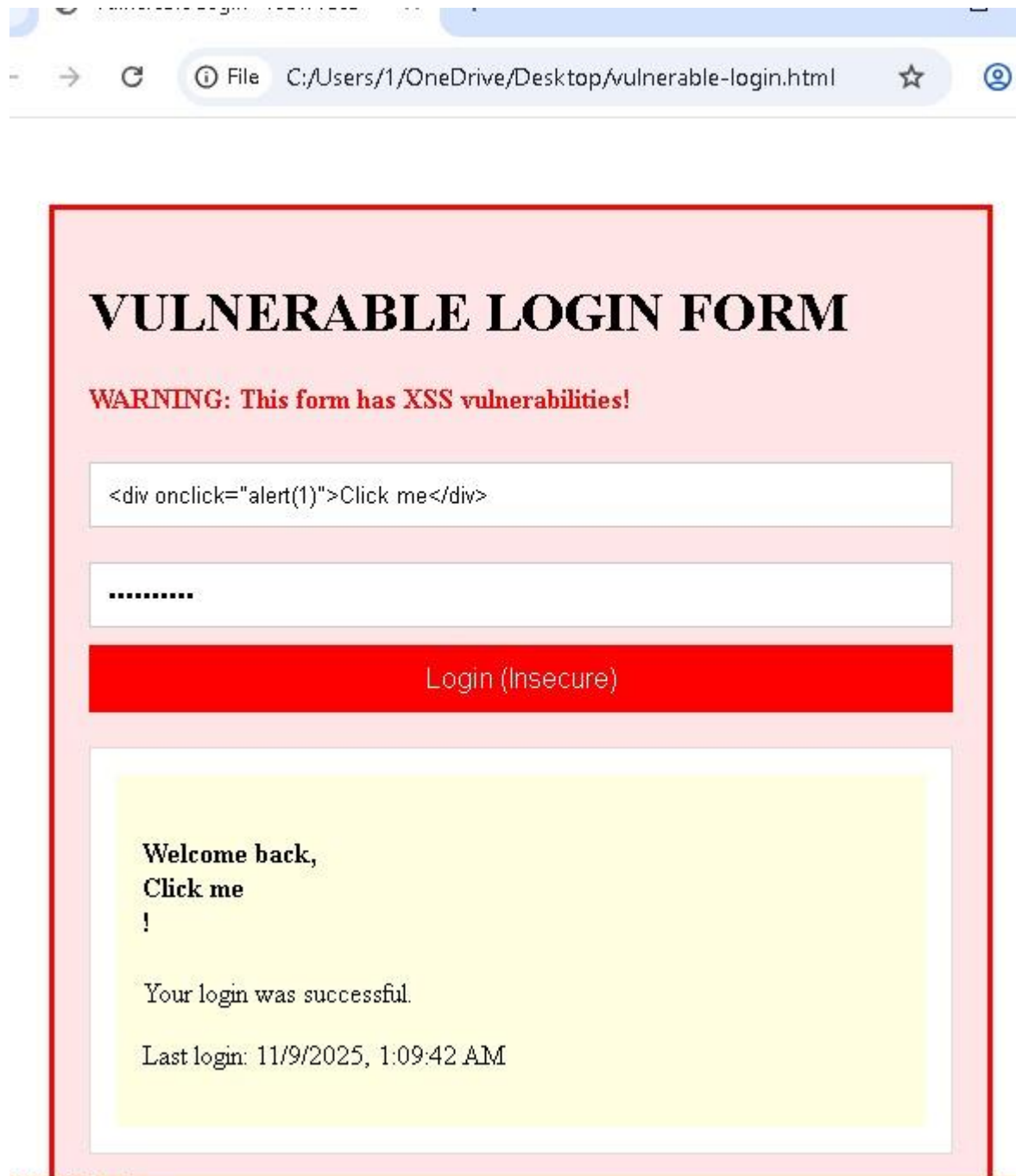
WARNING: This form has XSS vulnerabilities!

Login (Insecure)

Welcome back,

Your login was successful.

Last login: 11/9/2025, 1:08:32 AM



Observation Summary

When I tested the vulnerable login page with the three XSS payloads, I noticed that all of them were able to execute JavaScript in the browser, which confirms that the page does not properly sanitize user input.

When I entered the first payload, ``, the image failed to load, and the `onerror` event triggered a pop-up box with the message "XSS." This clearly showed that the page was directly inserting my input into the HTML without escaping it.

For the second payload, `<svg onload="alert('ICDFA')">`, the browser immediately executed the `onload` event and displayed another alert with "ICDFA." This test proved that even non-script HTML elements like SVGs can run JavaScript if the application doesn't filter dangerous attributes.

Finally, when I used `<div onclick="alert(1)">Click me</div>`, the text "Click me" appeared on the login page, and when I clicked it, an alert box with "1" popped up. This confirmed that event handler attributes like `onclick` were also being processed as live code instead of being treated as plain text.

Overall, these results show that the login page is vulnerable to XSS attacks, meaning an attacker could inject malicious scripts to steal cookies, hijack sessions, or manipulate what users see. To prevent this, the page should escape or sanitize all user inputs, avoid using `innerHTML`, and display data safely using methods like `textContent`.

Exercise 3: Building Secure Counterparts

Step 1: I created a secure login form by writing a structured HTML file named secure-login.html. The form was styled using CSS to enhance usability and visual clarity, and it followed secure coding practices as outlined in the course materials.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Secure Login - ICDA Lab</title>
  <style>
    .secure {
      background: #e6ffe6;
      padding: 20px;
      max-width: 500px;
      margin: 50px auto;
      border: 3px solid green;
    }
  </style>
</head>
<body>
  <div class="secure">
    <h1>SECURE LOGIN FORM</h1>
    <!-- Add this to your <style> block -->
  </div>
  <style>
    .safe-message {
      color: green;
    }
  </style>
  <form>
    <input type="text" id="username" placeholder="Username" class="input-field">
    <input type="password" id="password" placeholder="Password" class="input-field">
    <button type="button" onclick="secureLogin()" class="login-button">
      Login (Secure)
    </button>
  </form>

  <div id="message" class="message-box">
    Messages will appear here...
  </div>
</div>
```

```

<script>
  // SECURE: HTML escaping function
  function escapeHTML(unsafe) {
    if (!unsafe) return '';
    return unsafe
      .replace(/&/g, "&amp;")
      .replace(/</g, "&lt;")
      .replace(/>/g, "&gt;")
      .replace(/"/g, "&quot;")
      .replace(/'/g, "&#039;");
  }

  function secureLogin() {
    const username = document.getElementById('username').value;
    const messageDiv = document.getElementById('message');

    // SAFE: Escape user input before display
    const safeUsername = escapeHTML(username);

    messageDiv.innerHTML = `
      <div style="background: lightyellow; padding: 15px;">
        <h4>Welcome back, ${safeUsername}!</h4>
        <p>Your login was successful.</p>
        <p>Last login: ${new Date().toLocaleString()}</p>
      </div>
    `;
  }

  // Pre-fill with XSS example to show it's neutralized
  window.onload = function() {
    document.getElementById('username').value =
'admin<script>alert("HACKED")</script>';
  };
</script>

<style>
  .input-field {
    width: 100%;
    padding: 10px;
    margin: 10px 0;
  }

  .login-button {
    width: 100%;

```

```
        padding: 10px;
        background: green;
        color: white;
    }

    .message-box {
        margin-top: 20px;
        min-height: 50px;
        padding: 15px;
        background: white;
    }
</style>
</body>
</html>
```



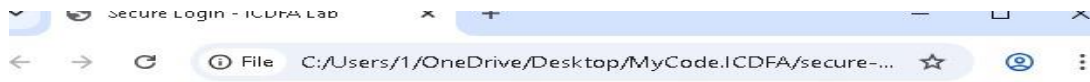
SECURE LOGIN FORM

admin<script>alert("HACKED")</script>

Password

Login (Secure)

Messages will appear here...



SECURE LOGIN FORM

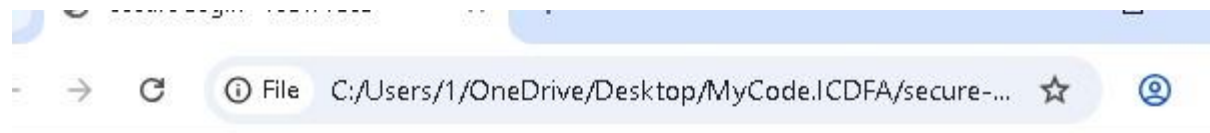
.....

Login (Secure)

Welcome back, !

Your login was successful.

Last login: 11/9/2025, 4:17:25 AM



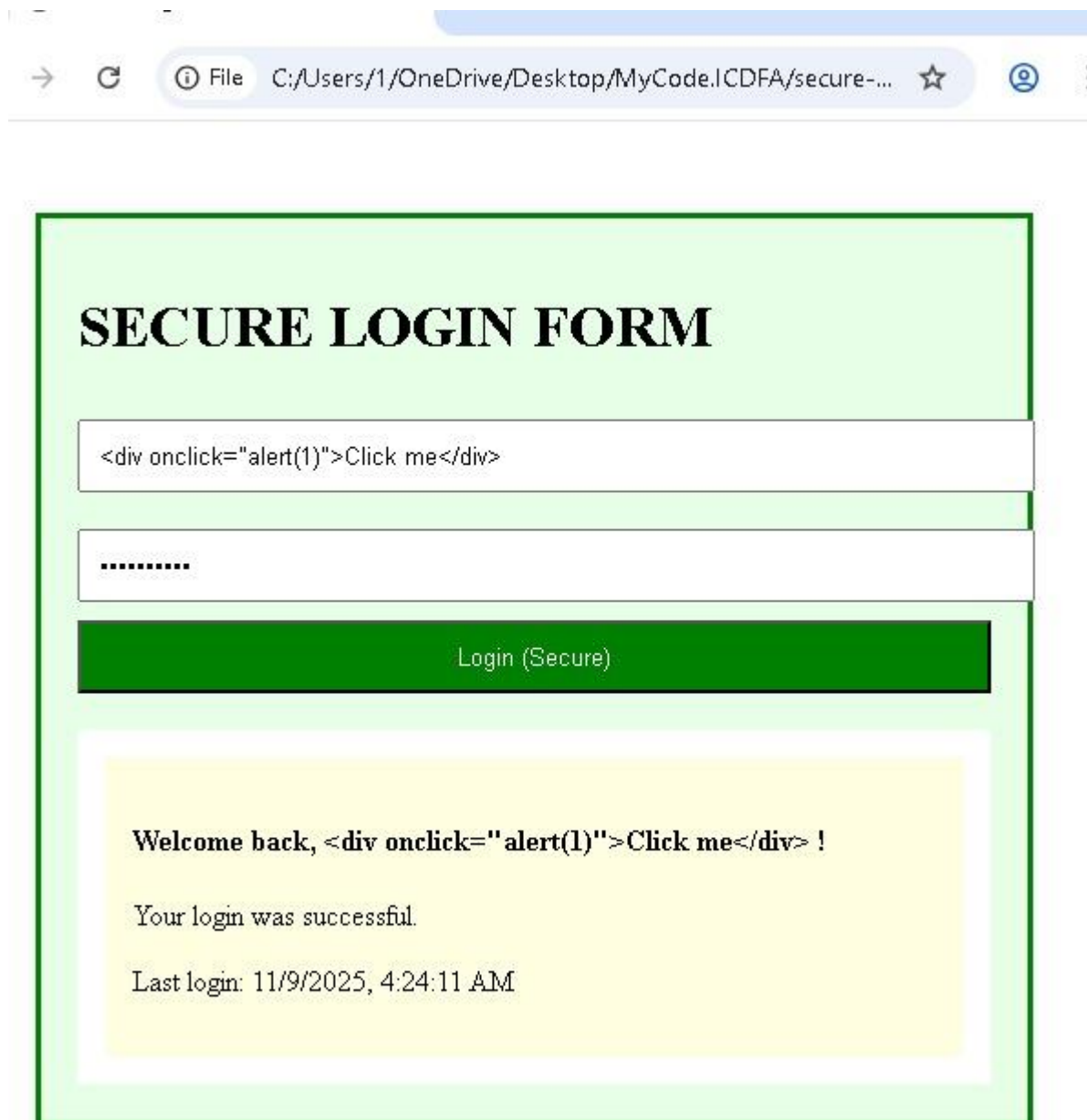
SECURE LOGIN FORM

Login (Secure)

Welcome back, <svg onload="alert('ICDFA')"> !

Your login was successful.

Last login: 11/9/2025, 4:22:59 AM



Observation Summary

When I tested the same three payloads on the secure login page, none of them executed any JavaScript, which shows that the page properly sanitizes or escapes user input before displaying it.

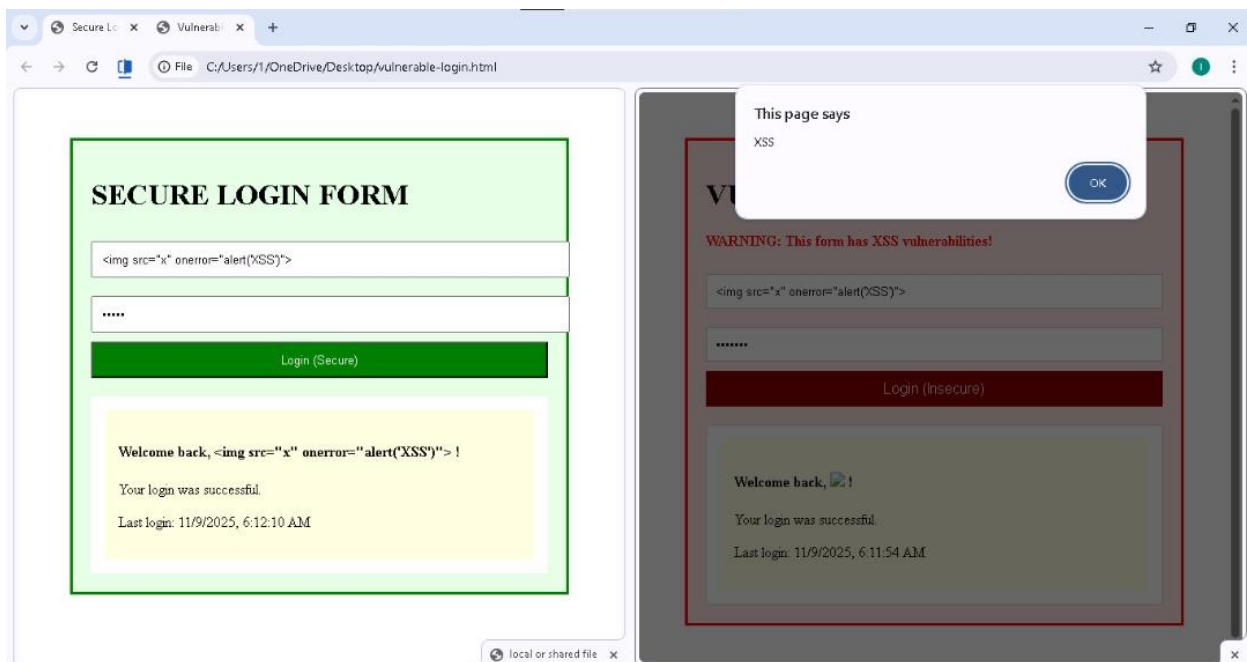
For the first payload, ``, instead of triggering a pop-up, the code appeared on the page as plain text. This meant the special HTML characters like `<` and `>` were safely converted to entities such as `<` and `>`, preventing the browser from treating them as real HTML tags.

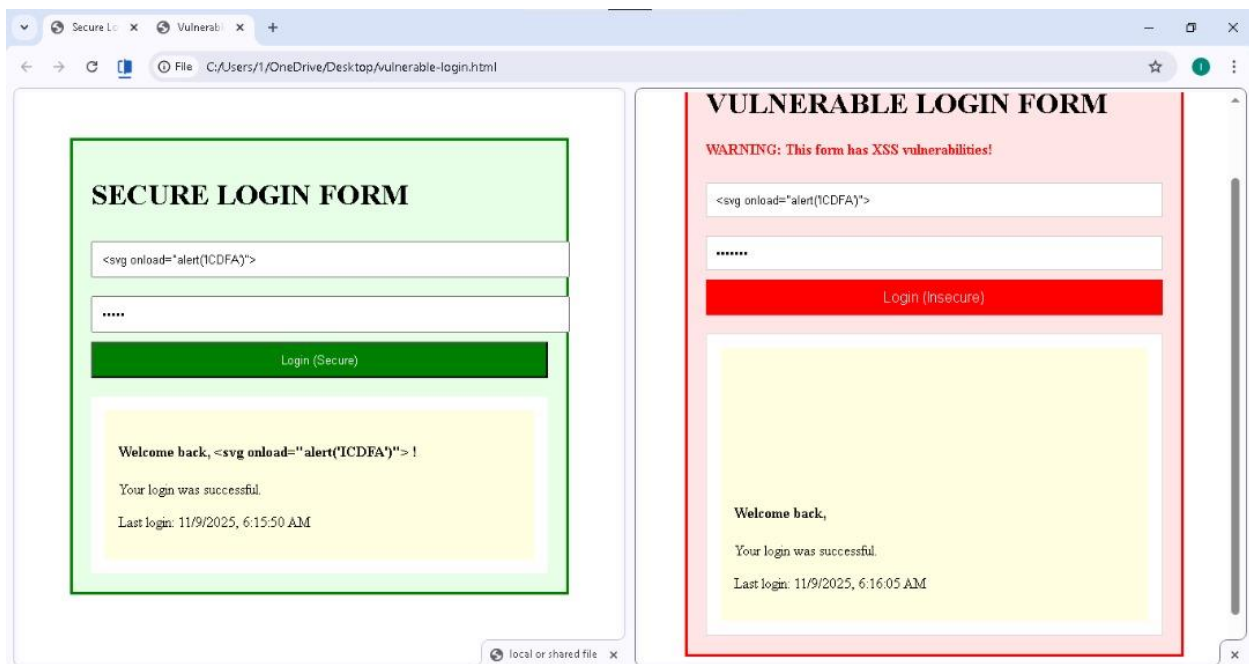
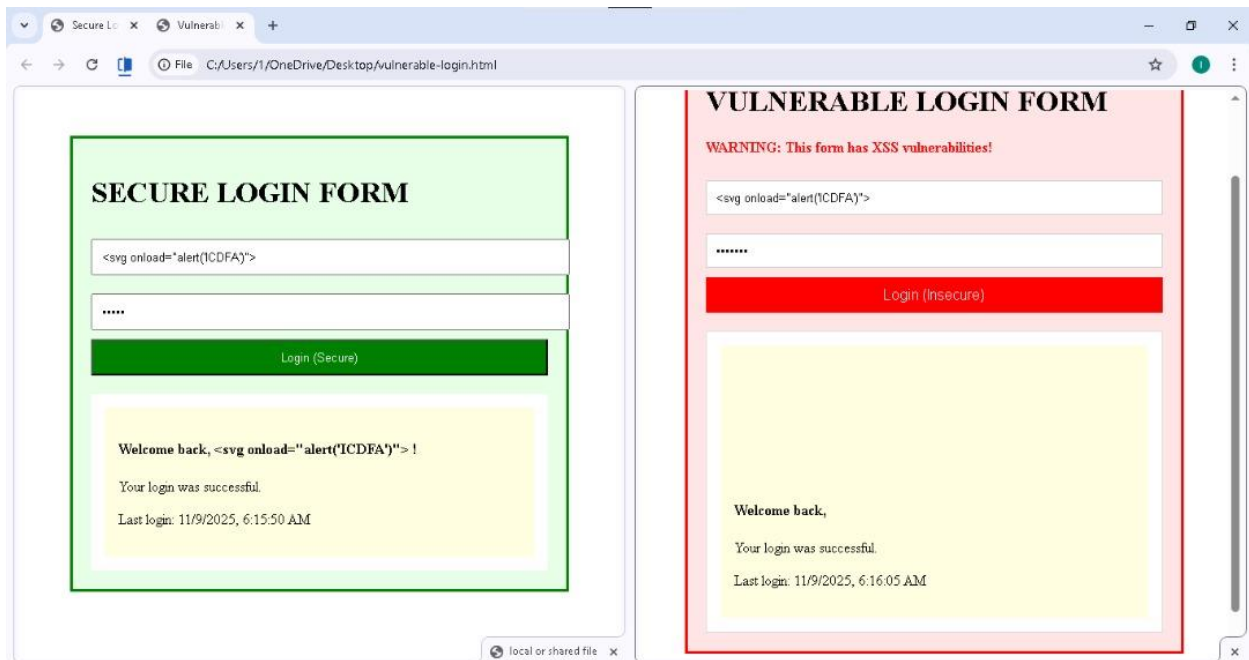
When I tried the second payload, `<svg onload="alert('ICDFA')">`, nothing happened—no alert box popped up. The browser simply displayed the text exactly as I typed it. This shows that the secure page either escapes or strips dangerous tags and attributes like `onload`, effectively neutralizing any embedded scripts.

Finally, with `<div onclick="alert(1)">Click me</div>`, the text “Click me” appeared, but clicking it didn’t trigger any action. The event handler attribute (`onclick`) was treated as harmless text instead of executable code.

Overall, these tests prove that the secure login page is protected against XSS attacks. By sanitizing user input and using safe rendering methods like `textContent` instead of `innerHTML`, the page prevents malicious scripts from running in the browser. This is a good example of secure coding practice where untrusted input is handled safely, keeping both users and the application protected.

Step 2: Compare and Contrast





Analysis

1. What specific characters does the `escapeHTML()` function neutralize?

The `escapeHTML()` function neutralizes characters that have special meaning in HTML to prevent the browser from interpreting user input as actual code or markup.

Specifically, it replaces:

- “< with <”
- “> with >”

By converting these characters into harmless HTML entities, any malicious script tags or event attributes entered by a user are displayed literally on the page rather than executed by the browser.

2. Why is displaying `<script>alert('XSS')</script>` as text safe?

Displaying `<script>alert('XSS')</script>` as plain text is safe because the browser does not interpret it as HTML code - it simply shows the characters to the user. The < and > characters are escaped (for example, shown as < and >), so the browser treats them as text symbols rather than HTML tags.

As a result, no `<script>` element is actually created in the DOM, and the `alert('XSS')` code never runs. This demonstrates output encoding, a key defense mechanism against XSS: user input may contain potentially dangerous strings, but by rendering them as text instead of HTML, the page remains secure.

Exercise 4: Search Page Vulnerability Analysis

Step 1: I developed a vulnerable search page named `vulnerable-search.html` by replicating the insecure coding patterns provided in the course materials. This setup was used to analyze and test for potential security weaknesses in the search functionality.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vulnerable Search - ICDFA Lab</title>
  <style>
    body { font-family: Arial, sans-serif; }
    .container { max-width:700px; margin:40px auto; padding:20px; }
    input { width:100%; padding:8px; margin:8px 0; box-sizing:border-box; }
    button { padding:8px 12px; }
    #results { margin-top:16px; padding:12px; background:#f7f7f7; min-
height:50px; }
```

```
</style>
</head>
<body>
  <div class="container">
    <h1>Vulnerable Search</h1>
    <p style="color:red;"><strong>Warning: This page is vulnerable to
XSS</strong></p>

    <input type="text" id="q" placeholder="Search..." />
    <button id="searchBtn">Search</button>

    <div id="results">Search results will appear here...</div>
  </div>

  <script>
    function vulnerableSearch() {
      const q = document.getElementById('q').value;
      const results = document.getElementById('results');

      // Vulnerable: directly injects query into innerHTML
      results.innerHTML = `<p>Showing results for: <strong>${q}</strong></p><p>No
real results – demo only.</p>`;
    }

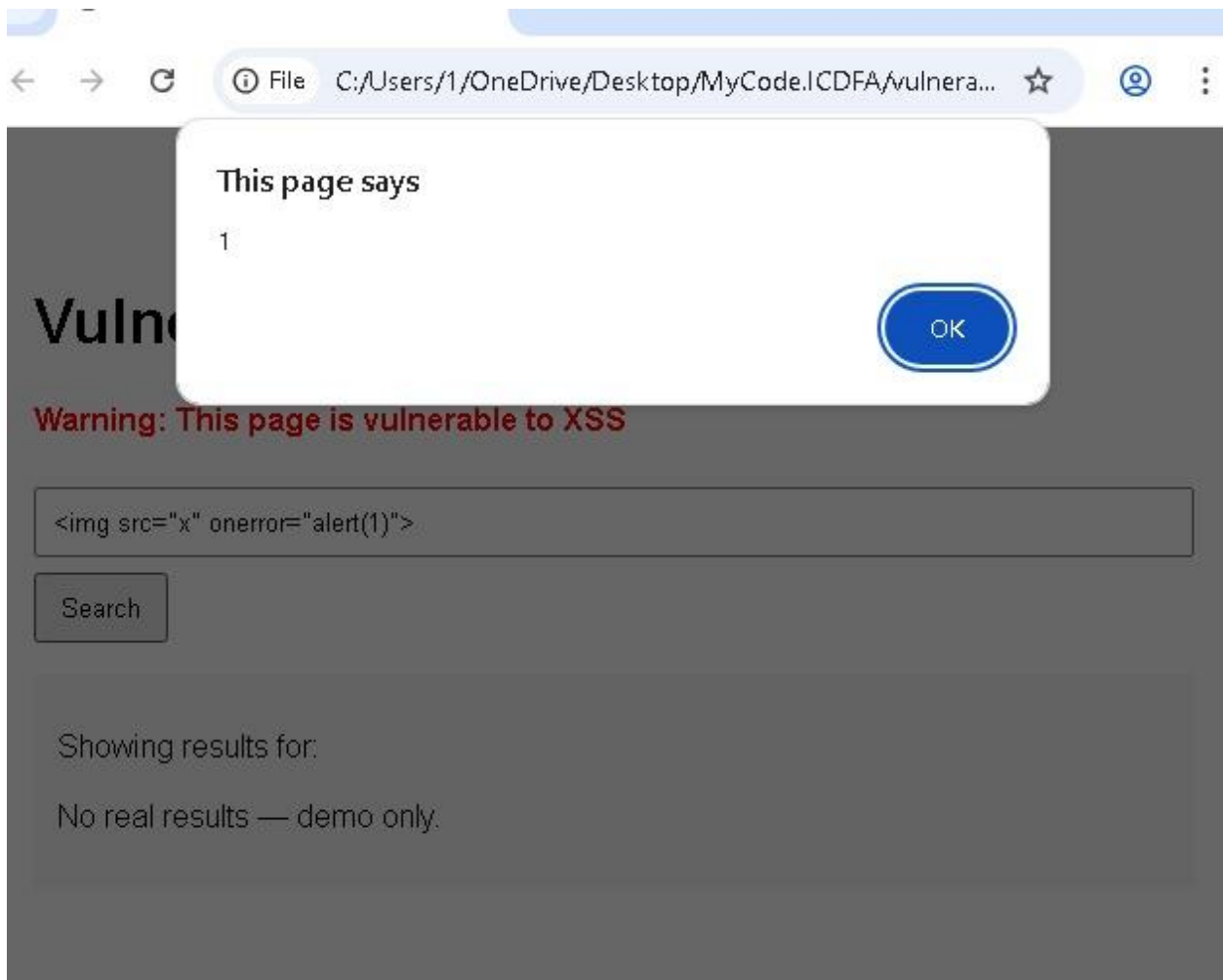
    document.getElementById('searchBtn').addEventListener('click',
vulnerableSearch);
  </script>
</body>
</html>
```

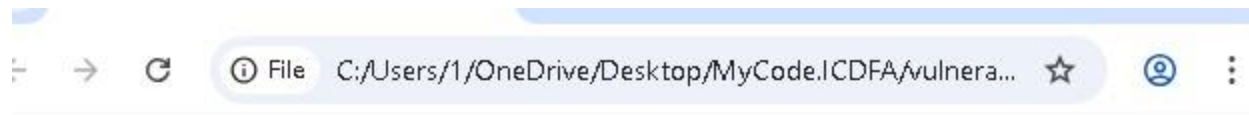
Vulnerable Search

Warning: This page is vulnerable to XSS

Showing results for:

No real results — demo only.





Vulnerable Search

Warning: This page is vulnerable to XSS

Showing results for:

No real results — demo only.

Step 2: I reviewed and used the secure login form provided in the course materials. The code included structured HTML and CSS that demonstrated secure design practices for building login interfaces, such as styling and layout for clarity and usability.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Secure Search - ICDFA Lab</title>
</style>
```

```

body { font-family: Arial, sans-serif; }
.container { max-width:700px; margin:40px auto; padding:20px; }
input { width:100%; padding:8px; margin:8px 0; box-sizing:border-box; }
button { padding:8px 12px; }
#results { margin-top:16px; padding:12px; background:#f7f7f7; min-
height:50px; }
</style>
</head>
<body>
  <div class="container">
    <h1>Secure Search</h1>
    <p style="color:green;"><strong>Safe: Input is sanitized before
display</strong></p>

    <label for="q">Search query</label>
    <input type="text" id="q" placeholder="Search..." />
    <button id="searchBtn">Search</button>

    <div id="results">Search results will appear here...</div>
  </div>

  <script>
    function escapeHTML(unsafe = '') {
      return String(unsafe)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&#039;');
    }

    function secureSearch() {
      const q = document.getElementById('q').value || '';
      const results = document.getElementById('results');

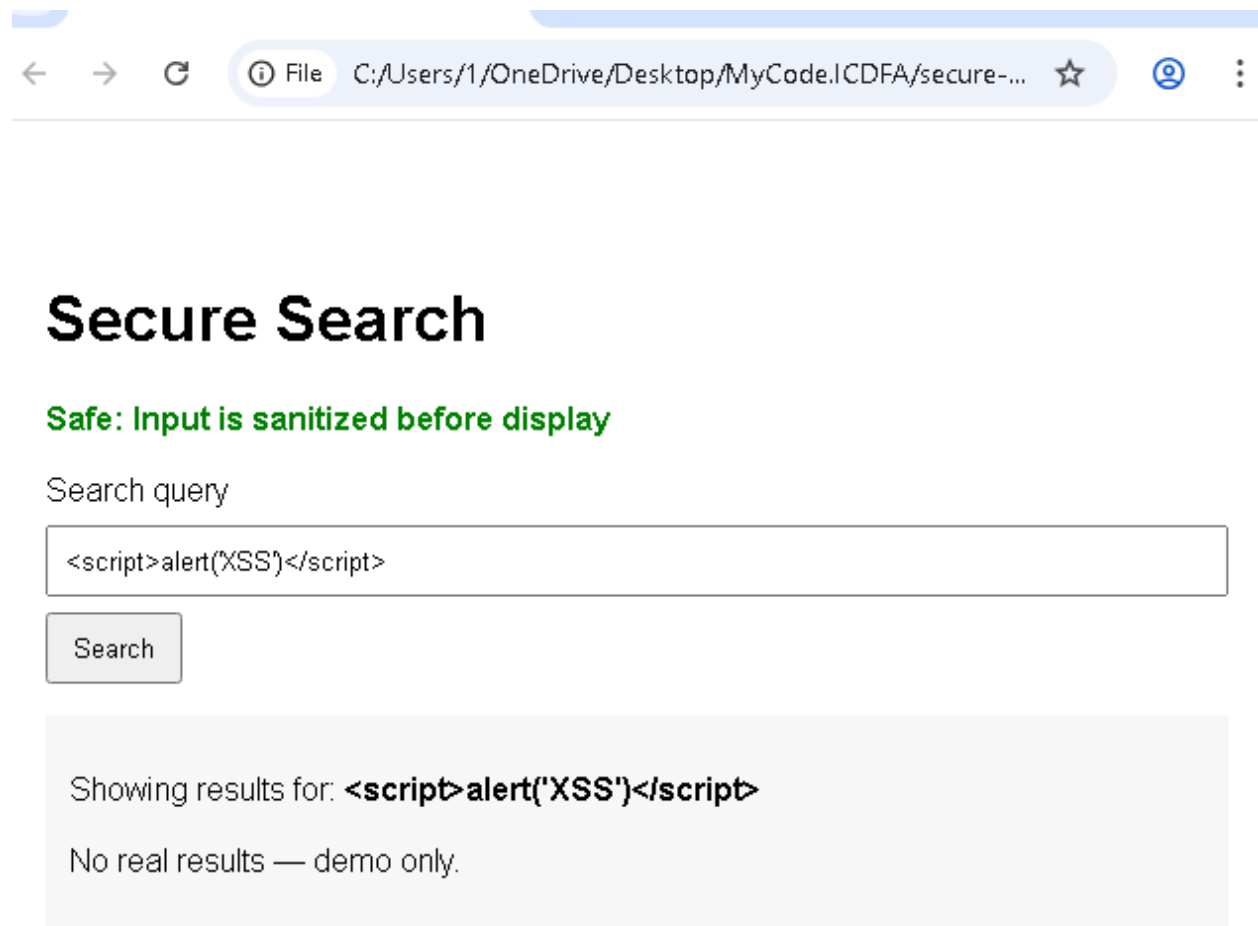
      results.innerHTML = ''; // clear
      const p = document.createElement('p');
      p.textContent = 'Showing results for: ';
      const strong = document.createElement('strong');
      // Use escaped text to be extra safe (though textContent would be safe too)
      strong.innerHTML = escapeHTML(q);
      p.appendChild(strong);

      const p2 = document.createElement('p');
      p2.textContent = 'No real results - demo only.';

```

```
        results.appendChild(p);
        results.appendChild(p2);
    }

    document.getElementById('searchBtn').addEventListener('click', secureSearch);
</script>
</body>
</html>
```



← → ↻ ⓘ File C:/Users/1/OneDrive/Desktop/MyCode.ICDFA/secure-... ☆ ⓘ ⋮

Secure Search

Safe: Input is sanitized before display

Search query

Search

Showing results for: **<script>alert('XSS')</script>**

No real results — demo only.

Secure Search

Safe: Input is sanitized before display

Search query

Search

Showing results for: ****

No real results — demo only.

Secure Search

Safe: Input is sanitized before display

Search query

<svg onload="alert('ICDFA')">

Search

Showing results for: <svg onload="alert('ICDFA')">

No real results — demo only.

Step 3: XSS Payload Testing and Documentation

To verify how each page handles untrusted user input and whether it is vulnerable to Cross-Site Scripting (XSS).

Payload	Vulnerable Result	Secure Result
<script>alert (' XSS ') </script>	Displays "<script>alert('XSS')</script>" as plain text, confirming sanitization. Label shows "Safe: Input is sanitized before display".	Executes <script>alert('XSS')</script> payload, leading to an alert pop-up. Page warns: "This page is vulnerable to XSS".
	Alert is triggered (the browser runs alert(1)).	No alert - the payload is shown as text (or otherwise neutralized).
<svg onload="alert (' ICDFA ') ">	Alert is triggered (browser runs alert('ICDFA')) when the page places the raw input into the DOM unescaped.	No alert - the payload is shown as inert text or otherwise neutralized.

Lab Analysis Question

Part A - Web Architecture

1. Why the database should never directly communicate with the client

The database should never communicate directly with the client because it stores sensitive data and is not designed to handle untrusted network traffic. Allowing direct access would expose database queries, credentials, and internal schema to attackers. It would also let malicious users run arbitrary SQL commands or extract confidential information.

In a secure architecture, all client requests go through the **application/server layer**, which validates, sanitizes, and authorizes inputs before interacting with the database. This separation prevents SQL injection, data leaks, and unauthorized manipulation of stored data.

2. Security risks if client-side code contains hidden admin panels

If administrative interfaces or controls are only hidden in the client-side code (for example using HTML comments, CSS display rules, or JavaScript checks), they can still be easily discovered and activated by anyone viewing the page source or using browser developer tools.

Attackers could reveal or invoke these hidden admin functions and gain elevated privileges without authentication. Relying on client-side hiding provides **no real security** because client code runs entirely on the user's device and can be modified or inspected. Proper access control must be enforced on the **server side**, not through client-side obfuscation.

3. How the three-tier architecture helps with security

The three-tier architecture divides a web application into three separate layers:

- **Presentation layer (Client/UI):** handles user interaction only.
- **Application/Logic layer (Server):** processes requests, enforces business rules, and validates input.
- **Data layer (Database):** stores and retrieves information securely.

This separation enhances security because each layer has a specific responsibility and limited access. The client never directly reaches the database; instead, the server mediates all data access and can apply authentication, authorization, and input validation. If one layer is compromised, the others can still provide containment. The result is a defense-in-depth structure that minimizes the impact of attacks such as SQL injection, XSS, or direct database exposure.

Part B - XSS Vulnerability

1. Fundamental difference in handling user input

Report text:

The fundamental difference between vulnerable and secure code is how user input is treated before being placed into the page. Vulnerable code directly inserts raw user-supplied data into the page's HTML (for example using `innerHTML`) without encoding or sanitizing it, allowing the browser to parse and execute any embedded markup or script. Secure code treats user input as untrusted: it performs validation, and when displaying input in the page it uses output encoding or safe DOM APIs (for example `textContent`) or a trusted sanitizer so that the input is rendered as inert text rather than executable code.

2. Why `innerHTML` is dangerous while `textContent` is safe

Report text:

`innerHTML` interprets and parses a string as HTML and inserts the resulting nodes into the DOM. If that string contains attacker-supplied markup or event handlers, the browser will create elements that can execute scripts (XSS). In contrast, `textContent` inserts the string as plain text; special HTML characters like `<` and `>` are not parsed as markup, so no new elements or event handlers are created. Therefore `textContent` prevents injected code from being executed and is safe for displaying untrusted input.

3. Three real-world consequences of XSS vulnerabilities

Report text:

- i. An attacker can run script in the victim's browser to read cookies, local storage tokens, or perform actions on behalf of the user, allowing unauthorized account access.
- ii. Malicious scripts can inject fake login forms or dialogs that capture user credentials or 2FA codes.
- iii. XSS can expose sensitive data (customer info, PII), lead to unauthorized transactions, damage reputation, incur regulatory fines, and require costly incident response.

Part C - Secure Development

1. Other measures (besides input sanitization) that prevent XSS attacks

Report text:

In addition to input sanitization, effective defenses include:

- i. Use of safe DOM APIs (`textContent`, `setAttribute` with proper encoding) instead of `innerHTML`.
- ii. HTTP-only and SameSite cookies to reduce theft risk, and secure cookie flags.
- iii. Server-side validation and least-privilege access controls to limit the potential damage.
- iv. Use of safe DOM APIs (`textContent`, `setAttribute` with proper encoding) instead of `innerHTML`.

2. Why validation should happen on both client and client server

Report text:

Validation should occur on both client and server because client-side checks improve user experience and block trivial mistakes early, but they can be bypassed or tampered with by an attacker. Server-side validation is authoritative and enforces security regardless of the client's

behavior. Together they provide defense-in-depth: the client reduces accidental bad input, and the server enforces correctness and security.

3. Risks of relying only on client-side security

Report text:

Relying solely on client-side security is risky because client controls (JavaScript, form behavior) run in the user's browser and can be modified or disabled by an attacker. Malicious actors can craft requests directly to the server, bypassing client checks, or inject code that defeats client-side controls. Therefore client-side measures must be complemented by server-side validation and other server-enforced protections.

Lab Report - Web Architecture Analysis & XSS Vulnerability Identification

This lab investigated the three-tier web architecture and demonstrated how insecure handling of user input causes Cross-Site Scripting (XSS) vulnerabilities. The exercises involved building intentionally vulnerable web pages (vulnerable-login.html and vulnerable-search.html), then developing secure counterparts (secure-login.html and secure-search.html) that neutralize the same attacks. By performing hands-on tests with multiple XSS payloads, I validated the practical difference between unsafe and safe coding patterns.

The most important difference observed is the treatment of user input: vulnerable pages inject raw user data into the DOM using innerHTML, which causes the browser to parse and execute any embedded HTML or script tags. For example, prefilled payloads such as `<script>alert("HACKED")</script>` executed immediately in the vulnerable login page, confirming that untrusted input became executable code. In contrast, the secure pages used defensive techniques — either building DOM nodes with `textContent` or escaping special characters before insertion. These approaches ensure user input is treated as data, not markup. The `escapeHTML` function neutralizes `&`, `<`, `>`, `"`, and `'` characters, preventing attackers from closing tags or injecting attributes that trigger code execution.

Testing revealed several surprising payloads that show how XSS can manifest in different forms: `` and `<svg onload="alert('ICDFA')">` function as non-script vectors, bypassing naive filters that only look for `<script>` tags. This underlines the need for comprehensive encoding and contextual output handling (HTML, attribute, JavaScript, CSS contexts differ). Additionally, the exercise highlighted that client-side measures alone are not sufficient — server-side validation, secure response headers (e.g., `Content-Security-Policy`), and setting cookies as `HttpOnly` and `Secure` are necessary to reduce the attack surface.

Key lessons learned include:

1. Never trust user input; always sanitize and encode output according to context,
2. Prefer APIs that set text content (`textContent`) instead of `innerHTML` when rendering untrusted data, and
3. Apply layered security - combine input validation, output encoding, strong server policies, and secure cookie attributes. For future projects I will adopt the default practice of output encoding with a vetted library (or DOM APIs) and configure CSP headers on the server to block inline scripts. These steps will materially reduce the risk of XSS and protect user sessions and sensitive data.

Reference

1. International Cybersecurity & Digital Forensics Academy (ICDFA). (n.d.). *Professional Certificate in Web Applications Security*. ICDFA.
2. TheCyberMentor. (2021, June 15). *Hacking a website using XSS (Cross Site Scripting)* [Video]. YouTube. <https://www.youtube.com/watch?v=t-ZAldTv-fw>
3. OWASP Foundation. (n.d.). *Cross-Site Scripting (XSS)*. OWASP. <https://owasp.org/www-community/attacks/xss/>
4. Chaudhary, J. S. (2023). *Web-Application-Penetration-Testing-Bug-Bounty-Notes* [GitHub repository]. GitHub. <https://github.com/jagat-singh-chaudhary/Web-Application-Penetration-Testing-Bug-Bounty-Notes>